

RTL Design Sherpa

Delta Generator Architecture Specification 1.0

October 13, 2026

Table of Contents

1 Delta Gas Index....12

2 Delta Hardware Architecture Specification Index....13

2.1 Overview....13

2.2 Related Modules....14

2.3 Navigation....15

2.3.1 Front Matter....15

2.3.2 Chapter 1: Introduction....15

2.3.3 Chapter 2: System Overview....15

2.3.4 Chapter 3: Architecture....15

2.3.5 Chapter 4: Interfaces....15

2.3.6 Chapter 5: Performance....15

2.3.7 Chapter 6: Integration....15

3 Delta Micro-Architecture Specification Index....17

3.1 Overview....18

3.1.1 Key Features....18

3.1.2 Companion Specifications....18

3.2 Navigation....19

3.2.1 Front Matter....19

3.2.2 Chapter 1: Introduction....19

3.2.3 Chapter 2: Block Descriptions....19

3.2.4 Chapter 3: Cycle-Level Behavior....19

3.2.5 Chapter 4: Generated RTL....19

3.2.6 Chapter 5: Verification....19
4 Delta: AXI-Stream Crossbar Generator - Product Requirements Document....20
4.1 Executive Summary....21
4.2 1. Project Goals....22
4.2.1 1.1 Primary Goals....22
4.2.2 1.2 Non-Goals....22
4.3 2. Architecture....23
4.3.1 2.1 Flat Crossbar Topology....23
4.3.2 2.2 Tree Topology (retired 2026-10-12 — never implemented in the framework generator)....23
4.4 3. Functional Requirements....25
4.4.1 3.1 Code Generation....25
4.4.2 3.2 Performance Modeling....25
4.4.3 3.3 Protocol Compliance....25
4.5 4. Non-Functional Requirements....27
4.5.1 4.1 Performance Targets....27
4.5.2 4.2 Resource Targets....27
4.5.3 4.3 Code Quality....27
4.6 5. Interface Specifications....28
4.6.1 5.1 AXI-Stream Master Interface (S_AXIS)....28
4.6.2 5.2 AXI-Stream Slave Interface (M_AXIS)....28
4.6.3 5.3 Clock and Reset....29
4.7 6. Key Design Decisions....30
4.7.1 6.1 Why AXI-Stream vs AXI4?....30

4.7.2 6.2 Why Round-Robin vs Fixed Priority?.....30
4.7.3 6.3 Why Flat AND Tree Topologies?.....30
4.7.4 6.4 Why Python Generator vs Manual RTL?.....30
4.8 7. Comparison to APB Crossbar.....31
4.8.1 7.1 Similarities (~95% Code Reuse).....31
4.8.2 7.2 Differences.....31
4.8.3 7.3 Migration Effort.....31
4.9 8. Use Cases.....32
4.9.1 8.1 Primary: RISC Cores + DSP Array.....32
4.9.2 8.2 Secondary: Educational Demonstration.....32
4.9.3 8.3 Future: Integration with RAPIDS.....32
4.10 9. Verification Strategy.....33
4.10.1 9.1 Generator Verification.....33
4.10.2 9.2 RTL Verification.....33
4.10.3 9.3 Model Validation.....33
4.11 10. Documentation Requirements.....34
4.11.1 10.1 Specifications.....34
4.11.2 10.2 Code Documentation.....34
4.12 11. Success Metrics.....35
4.12.1 11.1 Functional Metrics.....35
4.12.2 11.2 Performance Metrics.....35
4.12.3 11.3 Quality Metrics.....35
4.13 12. Timeline and Milestones.....36
4.13.1 Week 1: Generator and Models.....36

4.13.2 Week 2: RTL and Verification....36
4.13.3 Week 3: Integration and Documentation....36
4.14 13. Open Questions....37
4.15 14. Revision History....38
4.16 16. Documentation....39
4.17 Appendix A: Glossary....40
5 Claude Code Guide: Delta (AXI-Stream Crossbar Generator)....41
5.1 Quick Context....42
5.2 How It Actually Works (verified against rtl/delta_axis_flat_4x16.sv)....43
5.3 Repository Map....44
5.4 Critical Rules....45
5.5 Common Tasks....46
6 Document Information....47
6.1 Delta Generator Architecture Specification....47
6.2 Revision History....48
6.3 Document Purpose....49
6.4 Intended Audience....50
6.5 Related Documents....51
7 What the Generator Produces....52
7.1 Output artifacts....53
7.2 Architecture in the generated flat module....54
8 When to Regenerate....55
8.1 Standard regeneration workflow....56
8.2 Pre-generated vs custom....57

9 Generator Entry Points....58
9.1 Flat crossbar generator....58
9.2 Tree primitives generator....59
9.3 Programmatic API....60
10 CLI and Programmatic API....61
10.1 delta_generator.py CLI....61
10.2 complete_tree_generator.py CLI....62
10.3 Programmatic API....63
11 Parameter Semantics....64
11.1 Generation-time parameters....64
11.2 Elaboration-time parameters....65
12 AMBA 5, TSTRB, and Parity Options....66
12.1 TSTRB byte qualifiers....66
12.2 AMBA 5 sidebands (--amba5)....67
12.2.1 Parity rule....67
12.2.2 Who generates and who checks....68
12.2.3 Error flags....68
12.2.4 TWAKEUP steering....69
13 Validation Rules....70
13.1 CLI-level checks....71
13.2 Internal checks....72
14 Pipeline Overview....73
14.0.1 Figure 3.1: Delta Generator Pipeline....73
14.1 Stages....74

15 Request Decode 75
15.1 Request matrix 76
15.2 Out-of-range TDEST 77
16 Arbitration and Routing 78
16.1 Packet-atomic round-robin arbitration 78
16.2 Crossbar mux 79
16.3 Backpressure routing 80
17 Boundary Module Instantiation 81
17.1 Flat crossbar 82
17.2 Tree primitives 83
18 Directory Layout 84
19 Module Header and Parameters 85
19.1 Header comment 86
19.2 Port naming convention 87
20 Flat Crossbar 88
20.1 Internal structures 89
20.2 Optional features 90
20.2.1 Performance counters 90
20.2.2 Assertions 90
21 Tree Primitives 91
21.1 delta_split_1to2 92
21.2 delta_merge_2to1 93
21.3 Fan-out / fan-in trees 94
21.4 Filelist convention 95

22 Full Regeneration Rule 96
22.1 Regeneration command 97
22.2 Do not hand-edit generated files 98
23 Pre-Generated Fixtures 99
23.1 Custom variants 100
24 DV Suite 101
24.1 Test focus areas 102
24.2 Running the suite 103
24.3 Adding a new test 104
25 Lint Checks 105
25.1 Common concerns 106
25.2 Synthesis smoke test 107
26 Adding a New Topology 108
26.1 Tree topology notes 109
27 Adding a New Generator Option 110
27.1 Backward compatibility 111
27.2 Example: adding a QoS sideband 112
28 Adding AMBA 5 Sidebands 113
28.1 TWAKEUP design note 114
28.2 Check-signal design note 115
29 Testing an Extension 116
29.1 Minimal test plan 117
29.2 Example: adding a custom variant test 118
29.3 Commit discipline 119

List of Figures

Figure 3.1: Delta Generator Pipeline..... 73

List of Tables

Table 1: AMBA 5 check signals emitted by --amba5.....67
Table 2: Check-signal ownership per port.....68

1 Delta Gas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Delta Hardware Architecture Specification Index

2.1 Overview

Version: 1.0 **Date:** 2026-10-12 **Purpose:** High-level hardware architecture specification for the Delta AXI4-Stream crossbar generator

---

2.2 Related Modules

- **Delta MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **QUICK_START.md** - Hands-on generator guide

2.3 Navigation

Note: Every chapter below is one source file; the document build assembles the spec from these links.

2.3.1 Front Matter

- Document Information

2.3.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

2.3.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

2.3.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Protocol Support

2.3.5 Chapter 4: Interfaces

- AXI-Stream Base Interface
- AXI-Stream Rich Interface (TSTRB + AMBA5)
- Clock and Reset

2.3.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resources

2.3.7 Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy
- Synthesis and Implementation

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Delta Micro-Architecture Specification Index

Component: Delta (AXI4-Stream Crossbar Generator) **Version:** 1.0 **Date:** 2026-10-12 **Purpose:** Detailed micro-architecture specification for the generated Delta RTL

3.1 Overview

Delta is a Python generator that emits a single self-contained SystemVerilog module per configuration — a flat M×N AXI4-Stream crossbar routing `s_axis_*` masters to `m_axis_*` slaves. This is the internals document: the one you open when you need to know what the generated RTL actually does, block by block, rather than what the high-level spec says.

3.1.1 Key Features

- **Flat crossbar only:** Tree generation was retired on 2026-10-12.
- **TDEST-based routing:** No address map; the destination slave is taken directly from `s_axis_tdest`.
- **Packet-atomic round-robin arbitration:** A granted master holds the slave until `TLAST`.
- **Combinational data mux:** The grant is the registered stage; payload passes through combinatorially.
- **Optional richness:** `TSTRB` byte qualifiers, and AMBA 5 `TWAKEUP` with the IHI 0051B ch. 5 odd-parity check signals (`TDATACHK` … `TWAKEUPCHK`), checked at every port with sticky error flags (default off).
- **Optional counters:** Per-master and per-slave packet counters (disable with `--no-counters`).
- **Formal assertions:** `ifdef FORMAL` checks for one-hot grants and in-range TDEST.

3.1.2 Companion Specifications

- **Delta HAS** - Hardware Architecture Specification (high-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

3.2 Navigation

3.2.1 Front Matter

- Document Information

3.2.2 Chapter 1: Introduction

- Overview

3.2.3 Chapter 2: Block Descriptions

- Request Decode
- Per-Slave Round-Robin Arbiter
- Crossbar Multiplexer
- TREADY Steering
- TWAKEUP Steering (AMBA5)
- Interface Check Signals (AMBA5)
- Packet Counters
- Assertions

3.2.4 Chapter 3: Cycle-Level Behavior

- Multi-Master Contention
- Packet Interleave Prevention

3.2.5 Chapter 4: Generated RTL

- Module Structure
- Parameters and Logic Implications

3.2.6 Chapter 5: Verification

- Test Strategy

Last Updated: 2026-10-12 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Delta: AXI-Stream Crossbar Generator - Product Requirements Document

Project: Delta **Version:** 1.0 **Date:** 2025-10-18 **Status:** Active Development

---

4.1 Executive Summary

Delta is an AXI-Stream crossbar generator that produces parameterized RTL for routing data between multiple masters and multiple slaves. The name “Delta” follows the water/river theme (like RAPIDS) - deltas are where rivers split into multiple branches, analogous to crossbar routing.

Key Features: - Python-based RTL generation (similar to APB crossbar automation) - Single topology support: flat crossbar (tree topology retired 2026-10-12 — see QUICK_START.md) - Performance modeling (analytical + simulation) - Complete AXI-Stream protocol compliance - Educational focus with rigorous specifications

---

4.2 1. Project Goals

4.2.1 1.1 Primary Goals

1. **Code Generation Excellence**
   - Python generator produces clean, parameterized SystemVerilog
   - Reuses patterns from existing APB crossbar automation (~95% code reuse)
   - Single tool generates multiple topologies
2. **Performance Rigor**
   - Analytical models (closed-form latency/throughput)
   - Discrete event simulation (SimPy)
   - Resource estimation
   - Validation against RTL synthesis
3. **Educational Value**
   - Demonstrates specification-driven design
   - Shows code generation techniques
   - Compares topology trade-offs
   - Suitable for instruction on GitHub
4. **Integration Ready**
   - Works with existing CocoTBFramework
   - Compatible with RAPIDS subsystem
   - Supports RISC core + DSP array use case

4.2.2 1.2 Non-Goals

- Vendor-specific IP (using custom generators only)
- Protocol conversion (AXIS only, not AXI4/APB)
- Advanced routing algorithms (static TDEST-based only)
- Dynamic reconfiguration

4.3 2. Architecture

4.3.1 2.1 Flat Crossbar Topology

Description: Full MxN crossbar with per-slave arbiters

Characteristics: - **Latency:** 2 cycles (arbitration + output register) - **Throughput:** High aggregate (each slave independent) - **Resources:** ~1,920 LUTs for 4x16 @ 64-bit - **Best For:** Production systems requiring low latency

Block Diagram:

```
Masters (M)          Arbiters (N)         Slaves (N)

M0 ------+----------+-[Arbiter_S0]----- S0
         |          |
M1 ------+          +-[Arbiter_S1]----- S1
         |          |
M2 ------+          +-[Arbiter_S2]----- S2
         |          |
M3 ------+          +-[Arbiter_S15]---- S15

Request Matrix      Grant Matrix
(decode TDEST)      (round-robin)
```

4.3.2 2.2 Tree Topology (retired 2026-10-12 — never implemented in the framework generator)

Description: Hierarchical composition of 1:2 splitters and 2:1 mergers

Characteristics: - **Latency:** 4-6 cycles (multi-stage pipeline) - **Throughput:** Lower aggregate (bottleneck at root) - **Resources:** ~1,600 LUTs for 4x16 @ 64-bit (fewer LUTs but more instances) - **Best For:** Educational examples, demonstrating modularity

Block Diagram:

```
Masters     Mergers        Splitters       Slaves

M0 --+
     +-[2:1]--[1:2]--+-[1:2]--+- S0
M1 --+               |         +- S1
                     |
M2 --+               +-[1:2]--+- S2
     +-[2:1]--------+         +- S3
M3 --+

Tree depth: log2(N) stages
```

4.4 3. Functional Requirements

4.4.1 3.1 Code Generation

REQ-GEN-001: Python generator shall produce SystemVerilog RTL - **Input:** Command-line parameters (masters, slaves, data width, topology) - **Output:** Synthesizable SystemVerilog modules - **Verification:** Verilator lint clean, synthesis clean

REQ-GEN-002: Generator shall support flat crossbar topology (tree topology retired 2026-10-12 — see QUICK_START.md) **REQ-GEN-003:** Generator shall support optional TSTRB byte qualifiers (`--strb-width N`; default 0 = base signal set) so generated crossbars port-match `rtl/amba/axis4/*` (added 2026-10-12). **REQ-GEN-004:** Generator shall support optional AMBA 5 sidebands (`--amba5`): TWAKEUP steering — TDEST-decoded OR per slave, valid before TVALID — and the AXI5-Stream interface check signals of IHI 0051B ch. 5 (TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK; odd parity, spec widths). Payload checks are carried and checked on ingress, control checks are generated or checked per port, and each port has a sticky `*_parity_error`. This matches `rtl/amba/axis5/*` (added 2026-10-12; TPARITY/`--parity-width` replaced by the spec check signals for issue #102). Both options default off; default output is byte-identical to pre-2026-10-12 generations. - **Flat:** Single crossbar module - **Tree:** Hierarchical node composition - **Both:** ~~Generate both variants with `--topology both`~~ (retired 2026-10-12; flat only)

REQ-GEN-003: Generated RTL shall be parameterized - NUM_MASTERS, NUM_SLAVES (up to 32x256) - DATA_WIDTH (8, 16, 32, 64, 128, 256, 512, 1024 bits) - DEST_WIDTH, ID_WIDTH (auto-calculated)

REQ-GEN-004: Generator shall follow APB crossbar patterns - ~95% code reuse from APB automation - Same request generation, arbitration, mux patterns - Signal name mapping (pclk->aclk, psel->tvalid, etc.)

4.4.2 3.2 Performance Modeling

REQ-MODEL-001: Analytical model shall provide closed-form results - Latency calculation (cycles and nanoseconds) - Throughput estimation (transfers/cycle, Gbps) - Resource estimation (LUTs, FFs)

REQ-MODEL-002: Simulation model shall use discrete event simulation - SimPy-based cycle-accurate model - Statistical analysis (mean, percentiles) - Traffic pattern support (uniform, hotspot, localized)

REQ-MODEL-003: Models shall be validated against RTL - Compare analytical vs simulation results - Validate against synthesis reports - Document discrepancies

4.4.3 3.3 Protocol Compliance

REQ-PROTO-001: Generated RTL shall comply with AXI-Stream specification - TVALID/TREADY handshaking - TLAST packet boundaries - TDEST, TID, TUSER sideband signals

REQ-PROTO-002: Arbitration shall provide packet atomicity - Grant locked until TLAST - No packet interleaving - Fair round-robin arbitration

REQ-PROTO-003: Backpressure shall propagate correctly - TREADY from slave to granted master - No deadlocks - No data loss

---

4.5 4. Non-Functional Requirements

4.5.1 4.1 Performance Targets

NFR-PERF-001: Flat crossbar latency <= 2 cycles **NFR-PERF-002:** Flat crossbar Fmax >= 300 MHz (UltraScale+) **NFR-PERF-003:** ~~Tree topology latency <= 6 cycles~~ (retired with tree topology, 2026-10-12) **NFR-PERF-004:** Throughput >= 0.7 transfers/cycle under mixed traffic

4.5.2 4.2 Resource Targets

NFR-RES-001: Flat 4x16 @ 64-bit <= 2,500 LUTs **NFR-RES-002:** Tree 4x16 @ 64-bit <= 2,000 LUTs **NFR-RES-003:** No BRAM usage (pure logic implementation)

4.5.3 4.3 Code Quality

NFR-QUAL-001: Generated RTL shall pass Verilator lint **NFR-QUAL-002:** Generator code shall follow PEP 8 style **NFR-QUAL-003:** Comprehensive inline documentation **NFR-QUAL-004:** Assertions for formal verification

---

4.6 5. Interface Specifications

4.6.1 5.1 AXI-Stream Master Interface (S_AXIS)

Signal	Width	Direction	Description
`s_axis_tdata[m]`	DATA_WIDTH	Input	Data payload
`s_axis_tvalid[m]`	1	Input	Valid indicator
`s_axis_tready[m]`	1	Output	Ready (backpressure)
`s_axis_tlast[m]`	1	Input	Packet boundary
`s_axis_tdest[m]`	DEST_WIDTH	Input	Target slave ID
`s_axis_tid[m]`	ID_WIDTH	Input	Transaction ID
`s_axis_tuser[m]`	USER_WIDTH	Input	User sideband

Array: `[NUM_MASTERS]` (one per master)

4.6.2 5.2 AXI-Stream Slave Interface (M_AXIS)

Signal	Width	Direction	Description
`m_axis_tdata[s]`	DATA_WIDTH	Output	Data payload
`m_axis_tvalid[s]`	1	Output	Valid indicator
`m_axis_tready[s]`	1	Input	Ready (backpressure)
`m_axis_tlast[s]`	1	Output	Packet boundary
`m_axis_tdest[s]`	DEST_WIDTH	Output	Target slave ID (pass-through)
`m_axis_tid[s]`	ID_WIDTH	Output	Transaction ID (pass-through)
`m_axis_tuser[s]`	USER_WIDTH	Output	User sideband (pass-through)

Array: `[NUM_SLAVES]` (one per slave)

4.6.3 5.3 Clock and Reset

Signal	Width	Direction	Description
`aclk`	1	Input	Clock
`aresetn`	1	Input	Active-low reset

4.7 6. Key Design Decisions

4.7.1 6.1 Why AXI-Stream vs AXI4?

Decision: Use AXI-Stream instead of full AXI4

Rationale: - Streaming workloads (RISC cores, DSP arrays) - Simpler protocol (no address phases, burst management) - Better fit for compute fabrics - Still allows future upgrade to AXI4 if needed

4.7.2 6.2 Why Round-Robin vs Fixed Priority?

Decision: Default to round-robin arbitration

Rationale: - Fair bandwidth allocation - No starvation - Suitable for general-purpose compute - Can add fixed-priority mode later if needed

4.7.3 6.3 Why Flat AND Tree Topologies?

Decision: Support both, not either/or

Rationale: - Flat for production (low latency, high throughput) - Tree for education (modular, demonstrates composition) - Python generator makes both easy (~zero extra cost) - Shows trade-off analysis

4.7.4 6.4 Why Python Generator vs Manual RTL?

Decision: Python-based code generation

Rationale: - Reuse existing APB crossbar automation (~95%) - Parameterization without manual edits - Consistent code quality - Educational value (code generation techniques) - Rapid design space exploration

---

4.8 7. Comparison to APB Crossbar

4.8.1 7.1 Similarities (~95% Code Reuse)

Component	APB Crossbar	Delta (AXIS)	Reusable?
Request generation	Address decode	TDEST decode	[PASS] Pattern same
Per-slave arbitration	Round-robin	Round-robin	[PASS] Identical logic
Grant matrix	MxN grants	MxN grants	[PASS] Identical
Data multiplexing	Mux PRDATA	Mux TDATA/TVALID/TLAST	[PASS] Same pattern
Backpressure	PREADY	TREADY	[PASS] Renamed only

4.8.2 7.2 Differences

Aspect	APB Crossbar	Delta (AXIS)
Address decode	Range check (base/end)	Direct TDEST (simpler!)
Packet concept	Single transaction	Multi-beat packets with TLAST
Arbitration	Re-arbitrate every cycle	Lock until TLAST (packet atomicity)
Signals	PRDATA, PSLVERR	TDATA, TVALID, TLAST, TDEST, TID, TUSER
Read/Write	Separate paths	Unified data path (simpler!)

4.8.3 7.3 Migration Effort

Estimated Time: ~75 minutes (from existing APB generator to AXIS)

Tasks: 1. Rename signals (10 min) 2. Simplify address decode to TDEST (5 min) 3. Add packet atomicity logic (15 min) 4. Add new signals (TLAST, TID, TUSER) to ports and mux (10 min) 5. Update module naming (5 min) 6. Test with 2x2 configuration (30 min)

4.9 8. Use Cases

4.9.1 8.1 Primary: RISC Cores + DSP Array

Scenario: 4 small RISC-V cores need to route compute tasks to 16 DSP accelerators

Configuration: - 4 masters (RISC cores) - 16 slaves (DSP arrays) - 64-bit data width - Flat topology (low latency critical)

Benefits: - Each RISC core can target any DSP (full flexibility) - Round-robin ensures fair DSP access - Low 2-cycle latency for task dispatch

4.9.2 8.2 Secondary: Educational Demonstration

Scenario: Teach students about interconnect design trade-offs

Configuration: - Generate flat crossbars (tree topology retired 2026-10-12) - Compare latency, throughput, resources - Show code generation techniques - Demonstrate verification methodology

Benefits: - Hands-on learning with working RTL - Clear performance comparisons - Demonstrates real-world design process

4.9.3 8.3 Future: Integration with RAPIDS

Scenario: Use Delta for compute fabric routing, RAPIDS for memory DMA

Configuration: - RAPIDS handles descriptor-based memory transfers - Delta routes compute tasks between processors - Protocol adapter (Network 2.0 <-> AXI-Stream)

Benefits: - Separate concerns (memory vs compute) - Reusable components - Scalable architecture

---

4.10 9. Verification Strategy

4.10.1 9.1 Generator Verification

Approach: 1. Unit tests for each generator function (Python unittest) 2. Lint generated RTL (Verilator) 3. Synthesis test (Vivado/Yosys) 4. Compare output for different parameters

Success Criteria: - All Python tests pass - Generated RTL lints clean - Synthesis completes without errors

4.10.2 9.2 RTL Verification

Approach: 1. CocoTB testbench framework (reuse AMBA patterns) 2. Test all MxS routing combinations 3. Concurrent traffic scenarios 4. Backpressure stress tests 5. Packet atomicity verification

Success Criteria: - 100% routing coverage - No deadlocks under stress - Packet atomicity confirmed - Performance matches model (±10%)

4.10.3 9.3 Model Validation

Approach: 1. Analytical model vs simulation model comparison 2. Simulation model vs RTL CocoTB results 3. Synthesis reports vs resource estimates

Success Criteria: - Latency within ±1 cycle - Throughput within ±15% - Resources within ±20%

4.11 10. Documentation Requirements

4.11.1 10.1 Specifications

PRD (this document) - Requirements, architecture, use cases

Technical Specification - Detailed block diagrams - Interface timing diagrams - Performance analysis

User Guide - Generator usage examples - Integration patterns - Best practices

4.11.2 10.2 Code Documentation

Generator Code: - Docstrings for all functions - Inline comments for complex logic - Usage examples in header

Generated RTL: - Module header with configuration - Block-level comments - Signal descriptions

Performance Models: - Algorithm explanations - Formula derivations - Validation methodology

---

4.12 11. Success Metrics

4.12.1 11.1 Functional Metrics

- [PASS] Generator produces lint-clean RTL
- [PASS] All routing combinations verified
- [PASS] No protocol violations
- [PASS] Packet atomicity enforced

4.12.2 11.2 Performance Metrics

- [PASS] Flat crossbar: 2-cycle latency
- [PASS] Tree topology: <=6-cycle latency
- [PASS] Throughput: >=0.7 transfers/cycle
- [PASS] Fmax: >=300 MHz

4.12.3 11.3 Quality Metrics

- [PASS] Code reuse: >=90% from APB generator
- [PASS] Model accuracy: ±10% vs RTL
- [PASS] Resource estimate: ±20% vs synthesis
- [PASS] Documentation: Complete and clear

4.13 12. Timeline and Milestones

4.13.1 Week 1: Generator and Models

- [PASS] Python generator (flat topology)
- [PASS] Analytical performance model
- [PASS] Simulation model (SimPy)
- [PASS] Specifications

4.13.2 Week 2: RTL and Verification

- Generate RTL variants (flat 4x16; tree variant retired 2026-10-12)
- CocoTB testbench framework
- Basic functional tests
- Performance validation

4.13.3 Week 3: Integration and Documentation

- RISC + DSP integration example
- Complete user guide
- Performance comparison report
- Educational materials

4.14 13. Open Questions

Q1: Should we add configurable FIFO insertion for burst buffering? - **Status:** Deferred to v1.1 (keep v1.0 simple)

Q2: Support for weighted round-robin arbitration? - **Status:** Deferred (default round-robin sufficient for now)

Q3: Integration with existing APB crossbar generator (unified tool)? - **Status:** Under consideration (separate for now, may merge later)

4.15 14. Revision History

Version	Date	Author	Changes
1.0	2025-10-18	RTL Design Sherpa	Initial release

4.16 16. Documentation

The aspirational DELTA mesh NoC specification was split out of this component on 2026-10-12 and lives at `projects/components/noc-ip/delta_spec/` (agent guide: `projects/components/noc-ip/CLAUDE.md`). To render it:

```
cd projects/components/noc-ip
./generate_pdf.sh --rev 0.25
```

Rendered DOCX/PDF land in `noc-ip/delta_spec/`. This component — the AXIS crossbar generator — is documented by README.md, QUICK_START.md, and `delta_index.md`.

4.17 Appendix A: Glossary

- **AXIS:** AXI-Stream (streaming variant of AXI protocol)
- **Delta:** Project name (river delta = branching flow, like crossbar routing)
- **Flat Topology:** Full MxN crossbar with all connections
- **Tree Topology:** Hierarchical composition of 1:2 and 2:1 nodes
- **Packet Atomicity:** Locking grant until TLAST (prevent interleaving)
- **TDEST:** Transaction destination (slave ID in AXI-Stream)
- **TID:** Transaction ID (master identifier in AXI-Stream)

END OF PRD

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Claude Code Guide: Delta (AXI-Stream Crossbar Generator)

Version: 1.0 **Last Updated:** 2026-10-12 **Purpose:** AI-specific guidance for the Delta component (fabric-gen-ip)

5.1 Quick Context

What: Delta is a **Python-based AXI4-Stream (AXIS) crossbar generator**. It emits a single self-contained SystemVerilog module per configuration — a flat M×N stream crossbar routing `s_axis_*` masters to `m_axis_*` slaves.

Status: Active development. RTL generates and simulates. `dv/tests/` covers the `--amba5` example only (routing, TWAKEUP, check signals); the base flavor still has no simulation suite. Re-homed from `noc-ip/delta` on 2026-10-12 — the move did not change any behavior.

Your Role: Help users generate, parameterize, and validate AXIS crossbars; keep generated RTL and this documentation in sync.

Not this component: the aspirational DELTA mesh NoC design draft (routers, virtual channels, XY routing) lives separately at `projects/components/noc-ip/delta_spec/` — **no NoC RTL exists**. When “delta” comes up without qualification, this crossbar generator is what is meant.

5.2 How It Actually Works (verified against `rtl/delta_axis_flat_4x16.sv`)

- **Route decode:** the target slave comes straight from `s_axis_tdest` (`DEST_WIDTH` bits) — there is **no address map**.
- **Arbitration:** per-slave round-robin; the grant is **packet-atomic** — once granted, a master holds the slave until `TLAST`.
- **Data path:** `TDATA/TVALID/TLAST/TDEST/TID/TUSER` crossbar mux is **combinational**; the **grant** is the registered stage.
- **Backpressure:** `TREADY` from the granted slave is steered back to the granted master only.
- **Optional flavors (2026-10-12, default off, default output byte-identical):** `TSTRB` (`--strb-width`) byte qualifiers, and AMBA 5 (`--amba5`): `TWAKEUP` plus the IHI 0051B ch. 5 check signals `TDATACHK` … `TWAKEUPCHK` (odd parity, spec widths, no width option), so generated crossbars port-match `rtl/amba/axis4/*` and `rtl/amba/axis5/*`. TWAKEUP is TDEST-decoded (OR per slave) and may assert before TVALID. Payload checks are carried through the mux and checked on ingress; control checks are generated or checked per port; sticky `s_/m_parity_error` per port. An `--amba5` module instantiates `rtl/common/dataint_parity` – its filelist must `-f` include `rtl/common/filelists/dataint_parity.f`.
- **Optional:** packet counters.

Parameters on the generated module: `DATA_WIDTH`, `DEST_WIDTH`, `ID_WIDTH`, `USER_WIDTH`, `NUM_MASTERS`, `NUM_SLAVES`.

5.3 Repository Map

Path	What
`bin/delta_generator.py`	The generator (`DeltaConfig`, `DeltaFlatCrossbar`, CLI)
`bin/delta_performance_model.py`	Analytical + simulation performance model
`bin/complete_tree_generator.py`	Retired tree-topology prototype (kept for history)
`rtl/delta_axis_flat_4x16.sv`	Reference generated crossbar (checked in)
`rtl_test/*.sv`	Generated variants (2×4, 3×8, 2×16, 4×16)
`rtl_test/delta_axis_flat_2x4_strb_amba5.sv`	Rich-flavor example (TSTRB + TWAKEUP + T*CHK check signals); filelist `rtl_test/filelists/delta_axis_flat_2x4_strb_amba5.f`
`dv/tests/`, `dv/tbclasses/`, `dv/tb/`	cocotb suite for the `--amba5` example (AXIS5 BFMs via a flattening harness)
`rtl/filelists/*.f`	Filelists (`$REPO_ROOT`-relative)
`README.md` / `PRD.md` / `QUICK_START.md`	Product docs (PRD REQ-GEN-002 is flat-only since 2026-10-12)
`delta_index.md`	Crossbar doc index
`TREE_TOPOLOGY_TEST_RESULTS.md`	Historical results for the retired tree path

5.4 Critical Rules

1. **Flat only.** `--topology tree` (and `both`) prints "not yet implemented in framework version". Do not promise tree generation; REQ-GEN-002 was amended 2026-10-12.
2. **Regenerate, don't hand-edit.** `rtl/delta_axis_flat_4x16.sv` and the `rtl_test/` variants are generator outputs. Fix `bin/delta_generator.py` and re-emit; note `--topology tree` was retired — regenerate with `flat`.
3. **AXIS only.** No AXI4/AXI4-Lite/APB conversion here (PRD non-goal). For memory-mapped fabrics see siblings `../bridge/` and `../lattice-apbx-xbar/`.
4. **Filelist paths** in `rtl/filelists/*.f` are `$REPO_ROOT`-relative — keep them working from any cwd.
5. **Do not resurrect the NoC** (`noc-ip/delta_spec/`) unless the owner explicitly asks for NoC work.

5.5 Common Tasks

```
# Generate a fresh crossbar
python3 bin/delta_generator.py --topology flat --masters 4 --slaves 16
\
    --data-width 64 --output-dir rtl/

# Rich flavor: TSTRB + AMBA 5 sidebands (matches rtl/amba/axis4 +
axis5).
# The generator names the file by shape (delta_axis_flat_2x4.sv), so
emit to a
# scratch dir and copy -- writing straight into rtl_test/ overwrites
the PLAIN
# 2x4 example. The flavored name carries the options.
python3 bin/delta_generator.py --topology flat --masters 2 --slaves 4
\
    --data-width 32 --strb-width 4 --amba5 --output-dir
/tmp/delta_amba5/
cp /tmp/delta_amba5/delta_axis_flat_2x4.sv
rtl_test/delta_axis_flat_2x4_strb_amba5.sv

# Its DV suite (AXIS5 BFMs, odd-parity checks on every port)
cd dv/tests && make clean-all && make run-all-func-parallel

# Performance comparison (historical flat/tree data retained)
python3 bin/delta_performance_model.py --topology compare
```

Maintained By: RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

6 Document Information

6.1 Delta Generator Architecture Specification

Property	Value
Document Title	Delta Generator Architecture Specification
Version	1.0
Date	2026-10-12
Status	Released
Classification	Open Source - MIT License

6.2 Revision History

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial GAS release — verified against `delta_generator.py`, `complete_tree_generator.py`, and emitted fixtures in `rtl/`.
1.1	2026-10-12	RTL Design Sherpa	AMBA 5 interface parity per IHI 0051B ch. 5 (issue #102): `TPARITY`/`PW_WIDTH`/`--parity-width` replaced by the odd-parity check signals `TDATACHK` … `TWAKEUPCHK` (spec widths, checked at every port, sticky `*_parity_error`), matching `rtl/amba/axis5`. First `dv/` suite.

6.3 Document Purpose

This Generator Architecture Specification (GAS) documents the Delta AXI-Stream crossbar code generator. It is written for the engineer who must re-run the generator, add a new topology or option, or understand why a generated module looks the way it does.

For the generated hardware, see the **Delta HAS** and **MAS**. For product requirements, see **PRD.md**.

6.4 Intended Audience

- RTL engineers extending the generator
- Verification engineers regenerating fixtures
- Integrators choosing pre-generated vs custom variants

6.5 Related Documents

- **Delta HAS** - Hardware Architecture Specification
- **Delta MAS** - Micro-Architecture Specification
- **PRD.md** - Product requirements
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

7 What the Generator Produces

The Delta generator (`bin/delta_generator.py`) emits a parameterized AXI-Stream flat crossbar module. A companion script (`bin/complete_tree_generator.py`) emits tree-structured primitives (1:2 splitters, 2:1 mergers, fan-out trees, and fan-in trees). All output is single-file SystemVerilog with no post-processing required.

7.1 Output artifacts

Artifact	Description
`delta_axis_flat_<M>x<N>.sv`	Core flat crossbar module.
`delta_split_1to2.sv`	1:2 splitter primitive (when `--nodes` is used).
`delta_merge_2to1.sv`	2:1 merger primitive from `complete_tree_generator.py`.
`delta_fanout_1to<N>.sv`	Fan-out tree scaffold (partial implementation).
`delta_fanin_<N>to1.sv`	Fan-in tree scaffold (partial implementation).

7.2 Architecture in the generated flat module

The generated `delta_axis_flat_<M>x<N>` module contains:

1. **Request decode** — `TDEST` is decoded directly into a slave request matrix.
2. **Per-slave arbitration** — round-robin with packet atomicity (grant held until `TLAST`).
3. **Crossbar mux** — routes `TDATA`, `TVALID`, `TLAST`, `TDEST`, `TID`, `TUSER`, optional `TSTRB`, and with `--amba5` the payload check bits.
4. **Optional AMBA 5 logic** — `TWAKEUP` steering, plus the IHI 0051B ch. 5 check signals: payload checks on ingress, control checks generated or checked, and sticky per-port error flags. Built from `rtl/common/dataint_parity`.
5. **Backpressure logic** — `s_axis_tready` driven from `m_axis_tready` of the granted slave.
6. **Optional performance counters** — packet counts per master and per slave.
7. **Optional assertions** — one-hot grant and TDEST-range checks under `FORMAL`.

Tree generation is currently a scaffold; the flat crossbar is the production-ready artifact.

8 When to Regenerate

Regenerate the Delta fixtures when any of the following change:

- `bin/delta_generator.py`
- `bin/complete_tree_generator.py`
- `bin/rtl_generators/verilog/module.py` (the framework base class)
- The area regression flow or lint rules

8.1 Standard regeneration workflow

```
cd projects/components/fabric-gen-ip/delta/bin
python delta_generator.py --masters 2 --slaves 4 --output-dir ../rtl
python complete_tree_generator.py --type merger --output ../rtl
```

After regeneration, run the DV suite:

```
cd ../dv/tests
make clean-all && make run-all-gate
```

8.2 Pre-generated vs custom

Always check whether a pre-generated fixture exists before generating a custom one. The pre-generated fixtures are the regression baseline; custom variants are for one-off sizes.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

9 Generator Entry Points

9.1 Flat crossbar generator

```
python3 delta_generator.py --masters 2 --slaves 4 --data-width 64 --
output-dir ../rtl
```

CLI flags are documented in the configuration chapter.

9.2 Tree primitives generator

```
python3 complete_tree_generator.py --type merger --output ../rtl
python3 complete_tree_generator.py --type fanout --size 4 --
output ../rtl
python3 complete_tree_generator.py --type fanin --size 4 --output
../rtl
```

9.3 Programmatic API

```
from delta_generator import DeltaConfig, DeltaFlatCrossbar

cfg = DeltaConfig(
    topology="flat",
    num_masters=2,
    num_slaves=4,
    data_width=64,
    dest_width=2,
    id_width=1,
    user_width=1,
    strb_width=0,
    amba5=False,
    registered_outputs=True,
    arbiter_type="round_robin",
    add_performance_counters=True,
)
gen = DeltaFlatCrossbar(cfg)
gen.verilog("../rtl")
```

The `verilog()` method writes `delta_axis_flat_<M>x<N>.sv` into the requested directory using the `Module` framework's `write()` helper.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

10 CLI and Programmatic API

10.1 `delta_generator.py` CLI

```
python3 delta_generator.py --masters 2 --slaves 4 --data-width 64 --
output-dir ../rtl
```

Flag	Default	Description
`--topology`	`flat`	`flat`, `tree`, or `both`. Tree is not fully implemented in the framework version.
`--masters`	required	Number of master interfaces.
`--slaves`	required	Number of slave interfaces.
`--data-width`	64	`TDATA` width in bits.
`--user-width`	1	`TUSER` width.
`--strb-width`	0	`TSTRB` width; 0 omits `TSTRB` ports.
`--amba5`	False	Emit AMBA 5 `TWAKEUP`, the IHI 0051B ch. 5 odd-parity check signals and the per-port `*_parity_error` flags. Widths are fixed by the spec; requires a whole-byte `--data-width`.
`--output-dir`	`../rtl`	Destination directory.
`--nodes`	False	Also emit the `delta_split_1to2` primitive.
`--no-counters`	False	Disable the optional performance counters.

10.2 `complete_tree_generator.py` CLI

Flag	Description
`--type`	`merger`, `fanout`, or `fanin`.
`--size`	Number of inputs/outputs for fan-out/fan-in.
`--output`	Output directory (default `../rtl`).

10.3 Programmatic API

The preferred programmatic entry point is the `DeltaConfig` dataclass plus `DeltaFlatCrossbar`:

```
from delta_generator import DeltaConfig, DeltaFlatCrossbar

cfg = DeltaConfig(
    topology="flat",
    num_masters=2,
    num_slaves=4,
    data_width=64,
    dest_width=2,
    id_width=1,
    user_width=1,
    strb_width=8,
    amba5=True,
    registered_outputs=True,
    arbiter_type="round_robin",
    add_performance_counters=True,
)

gen = DeltaFlatCrossbar(cfg)
gen.verilog("../rtl")
```

`dest_width` and `id_width` are normally calculated from `num_slaves` and `num_masters` but may be overridden if the caller supplies them explicitly.

---

11 Parameter Semantics

11.1 Generation-time parameters

These are fixed in the emitted module name and structure.

Parameter	Effect in emitted RTL
`num_masters`	Number of `s_axis_*` input interfaces; sets `NUM_MASTERS` and `TID` width.
`num_slaves`	Number of `m_axis_*` output interfaces; sets `NUM_SLAVES` and `TDEST` width.
`data_width`	`TDATA` width; sets `DATA_WIDTH`.
`user_width`	`TUSER` width; sets `USER_WIDTH`.
`strb_width`	If > 0, adds `TSTRB` ports and sets `STRB_WIDTH`; otherwise omits them.
`amba5`	Adds `TWAKEUP`, the IHI 0051B ch. 5 check signals and the `s_/m_parity_error` flags; adds the derived check-width parameters.
`registered_outputs`	Reserved for future output-register insertion; currently affects header comments.
`arbiter_type`	Reserved for future arbiter selection; currently only round-robin is emitted.
`add_performance_counters`	Adds 32-bit packet counters per master and slave.

11.2 Elaboration-time parameters

The generated flat crossbar exposes the following parameters:

```
module delta_axis_flat_2x4 #(
    parameter int DATA_WIDTH = 64,
    parameter int DEST_WIDTH = 2,
    parameter int ID_WIDTH   = 1,
    parameter int USER_WIDTH = 1,
    parameter int NUM_MASTERS = 2,
    parameter int NUM_SLAVES  = 4
) (
```

When `strb_width > 0`, `STRB_WIDTH` is added. When `amba5=True`, `TDATACHK_W`, `TIDCHK_W`, `TDESTCHK_W` and `TUSERCHK_W` are added, plus `TSTRBCHK_W` with TSTRB. Each defaults to an expression of its payload width, so leave them at their defaults.

Parameter	Typical use
`DATA_WIDTH`	Match the datapath width of the surrounding AXI-Stream network.
`DEST_WIDTH`	Wide enough to encode `NUM_SLAVES - 1`.
`ID_WIDTH`	Wide enough to encode `NUM_MASTERS - 1`.
`USER_WIDTH`	Carry sideband metadata alongside each beat.

The master and slave counts are **not** elaboration-time parameters. A 2×4 module cannot be instantiated as 2×8. Generate a new module instead.

12 AMBA 5, TSTRB, and Parity Options

12.1 TSTRB byte qualifiers

By default Delta omits TSTRB to match the common AXI-Stream use case where every byte is valid. To emit byte-qualifier ports:

```
python3 delta_generator.py --masters 2 --slaves 4 --strb-width 8
```

The generator adds `s_axis_tstrb` and `m_axis_tstrb` arrays and muxes them alongside TDATA.

12.2 AMBA 5 sidebands (`--amba5`)

When `--amba5` is set, the generator adds `TWAKEUP` and the full AXI5-Stream interface-protection set of IHI 0051B chapter 5, `Check_Type = Odd_Parity_Byte_All`. It also adds a sticky error flag on every port. The signal names and widths match `rtl/amba/axis5/*`, so a generated crossbar connects directly to those modules.

`--amba5` needs a whole-byte `--data-width`, because `TDATACHK` has one bit per `TDATA` byte. The CLI rejects anything else.

AMBA 5 check signals emitted by `--amba5`

Check signal	Width (Table 5-1)	Width parameter	Covers
TDATACHK	DATA_WIDTH/8	TDATACHK_W	TDATA
TSTRBCHK	ceil(STRB_WIDTH/8) (only with `--strb-width`)	TSTRBCHK_W	TSTRB
TLASTCHK	1	–	TLAST
TIDCHK	ceil(ID_WIDTH/8)	TIDCHK_W	TID
TDESTCHK	ceil(DEST_WIDTH/8)	TDESTCHK_W	TDEST
TUSERCHK	ceil(USER_WIDTH/8)	TUSERCHK_W	TUSER
TVALIDCHK	1	–	TVALID
TREADYCHK	1	–	TREADY
TWAKEUPCHK	1	–	TWAKEUP

Each width parameter is derived in the generated module header, for example `parameter int TDATACHK_W = DATA_WIDTH / 8`. If you override a payload width at instantiation, the check ports follow it. There is no separate parity-width option, because the spec fixes the widths. The old `--parity-width` flag has been removed.

12.2.1 Parity rule

Odd parity: a check bit and the bits it covers always hold an odd number of ones. Check bit `[n]` covers bits `[8n+7:8n]`. When a signal is not a whole number of bytes, the top check bit covers the short remainder (IHI 0051B section 5.3). A one-bit check is the inversion of its signal. The crossbar computes and checks every bit with one `rtl/common/dataint_parity` instance (`CHUNKS=1`, `parity_type=0`). The generated file therefore needs `dataint_parity` in its compile closure: `-f $REPO_ROOT/rtl/common/filelists/dataint_parity.f`, as `rtl_test/filelists/delta_axis_flat_2x4_strb_amba5.f` does.

12.2.2 Who generates and who checks

Check-signal ownership per port

Port side	Signal	Crossbar action
master (s_axis_*), crossbar receives	payload *CHK	Checked against the payload while TVALID is high. Then carried to the granted slave port unchanged, by the same grant mux as TDATA.
master	TVALIDCHK, TWAKEUPCHK	Checked every cycle out of reset (check enable is ARESETn).
master	TREADYCHK	Generated from the TREADY the crossbar drives.
slave (m_axis_*), crossbar transmits	payload *CHK	Carried from the granted master. When no master is granted, they idle at all ones, the odd parity of the all-zero idle payload.
slave	TVALIDCHK, TWAKEUPCHK	Generated from the TVALID / TWAKEUP the crossbar drives.
slave	TREADYCHK	Checked every cycle out of reset.

The payload check bits are carried, not regenerated. A check bit that arrives bad at a master port therefore reaches the slave still bad, and both the crossbar and the downstream receiver can see it.

12.2.3 Error flags

s_parity_error[NUM_MASTERS] and m_parity_error[NUM_SLAVES] are sticky and are cleared only by aresetn. A master port's flag sets on a bad payload *CHK beat, a bad TVALIDCHK, or a bad TWAKEUPCHK. A slave port's flag sets on a bad TREADYCHK. The crossbar does not drop or alter a beat that fails its check. The flag reports the failure and the beat is still delivered.

12.2.4 TWAKEUP steering

TWAKEUP is treated as a hint that may arrive before TVALID. The generator decodes it directly from TDEST without waiting for a grant:

```
m_axis_twakeup[s] = OR over all m of (s_axis_twakeup[m] &&
s_axis_tdest[m] == s);
```

m_axis_twakeupchk[s] is generated from this OR.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

13 Validation Rules

The generator validates CLI arguments through `argparse` and additional checks inside `main()`.

13.1 CLI-level checks

Rule	Behavior
`--topology` must be `flat`, `tree`, or `both`.	Enforced by `choices`.
`--masters` and `--slaves` are required integers.	Enforced by `required=True`.
`--strb-width` must be >= 0.	Custom check; negative values raise an error.
`--amba5` requires `--data-width % 8 == 0`.	Custom check; `TDATACHK` is one bit per `TDATA` byte (IHI 0051B Table 5-1).

13.2 Internal checks

Rule	Rationale
`num_masters >= 1` and `num_slaves >= 1`	A crossbar needs at least one master and one slave.
`data_width > 0`	Required for meaningful `TDATA`.
`user_width >= 1`	`TUSER` is always at least one bit.
`dest_width = ceil(log2(num_slaves))`	`TDEST` must encode all slave indices.
`id_width = ceil(log2(num_masters))`	`TID` must encode all master indices.

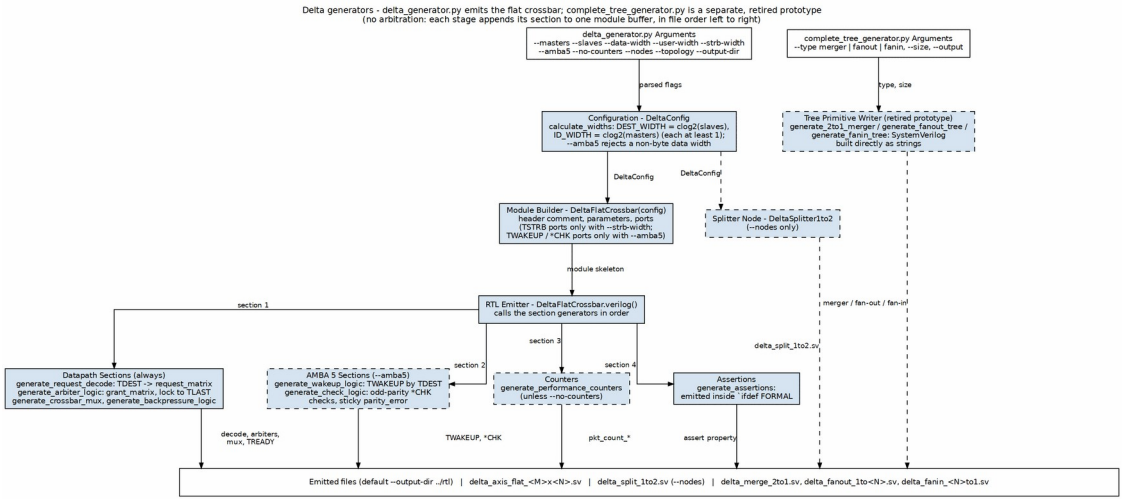
The generator does **not** currently validate that `strb_width` matches `data_width/8`; callers must ensure the width is consistent with their byte-qualifier policy.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

14 Pipeline Overview

The Delta generator uses the `Module` framework from `bin/rtl_generators/verilog/`. It builds the RTL as a sequence of method calls that append comments, instructions, and assignments to an internal buffer, then writes the completed module to disk.

14.0.1 Figure 3.1: Delta Generator Pipeline



Delta Generator Pipeline

The generator parses its flags into a `DeltaConfig`, builds the `DeltaFlatCrossbar` module skeleton, and `verilog()` appends the sections in file order: the always-present datapath, the AMBA 5 sections (`--amba5`), the counters (unless `--no-counters`) and the `FORMAL`-guarded assertions. `--nodes` also writes the 1:2 splitter. `complete_tree_generator.py`, the retired tree prototype, is a separate script that writes its primitives as strings.

14.1 Stages

1. **Argument parsing** — CLI flags become `DeltaConfig` fields.
2. **Width calculation** — `dest_width` and `id_width` are derived from `num_slaves` and `num_masters` unless overridden.
3. **Object construction** — `DeltaFlatCrossbar(config)` declares parameters and ports through the framework.
4. **RTL emission** — `verilog()` calls the generator methods in order:
   - `generate_header_comment()`
   - `generate_request_decode()`
   - `generate_arbiter_logic()`
   - `generate_crossbar_mux()`
   - `generate_backpressure_logic()`
   - `generate_wakeup_logic()` (AMBA 5 only)
   - `generate_check_logic()` (AMBA 5 only: odd-parity `*CHK` checks, sticky `parity_error`)
   - `generate_performance_counters()` (unless disabled)
   - `generate_assertions()`
5. **Framework finalization** — `start()` emits the module header; `end()` emits `endmodule`.
6. **File write** — `write()` saves `delta_axis_flat_<M>x<N>.sv`.

The `complete_tree_generator.py` script is independent and emits splitter/merger/tree scaffolds directly as strings.

---

15 Request Decode

Delta uses TDEST for routing instead of an address range. This is the fundamental difference between Delta and the APB crossbar generator.

15.1 Request matrix

The generator declares a per-slave request vector:

```
logic [NUM_MASTERS-1:0] request_matrix [NUM_SLAVES];
```

Each entry is populated in an `always_comb` block:

```
for (int s = 0; s < NUM_SLAVES; s++)
    request_matrix[s] = '0;

for (int m = 0; m < NUM_MASTERS; m++)
    if (s_axis_tvalid[m] && (s_axis_tdest[m] < NUM_SLAVES))
        request_matrix[s_axis_tdest[m]][m] = 1'b1;
```

Only masters with a valid beat and a `TDEST` inside the supported slave range contribute to the request matrix.

15.2 Out-of-range TDEST

The generator includes a formal assertion that flags `TDEST >= NUM_SLAVES`. The behavior in simulation for an out-of-range `TDEST` is undefined; the request is simply dropped. System-level validation must ensure that masters never present an invalid destination.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

16 Arbitration and Routing

16.1 Packet-atomic round-robin arbitration

Unlike the APB crossbar, which re-arbitrates every cycle, Delta locks the grant until the end of the current packet. This prevents interleaving beats from different masters on the same slave.

```
logic [NUM_MASTERS-1:0] grant_matrix [NUM_SLAVES];
logic [$clog2(NUM_MASTERS)-1:0] last_grant [NUM_SLAVES];
logic packet_active [NUM_SLAVES];
```

Arbitration state machine for each slave:

1. If `packet_active[s]` is true, hold the grant until `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]`.
2. If no packet is active and `request_matrix[s]` is non-zero, choose the next requesting master in round-robin order after `last_grant[s]`.
3. Set `packet_active[s]` when a new grant is awarded.

16.2 Crossbar mux

A per-slave `always_comb` block muxes the granted master's AXI-Stream signals to the slave output:

```
for (int m = 0; m < NUM_MASTERS; m++)
    if (grant_matrix[s][m]) begin
        m_axis_tdata[s]  = s_axis_tdata[m];
        m_axis_tvalid[s] = s_axis_tvalid[m];
        m_axis_tlast[s]  = s_axis_tlast[m];
        m_axis_tdest[s]  = s_axis_tdest[m];
        m_axis_tid[s]    = s_axis_tid[m];
        m_axis_tuser[s]  = s_axis_tuser[m];
        // TSTRB included when enabled; with --amba5 the payload check
        // bits (TDATACHK..TUSERCHK) ride the same grant, carried
unchanged
    end
```

16.3 Backpressure routing

Each master's `s_axis_tready` is driven from the `m_axis_tready` of the slave to which it is currently granted:

```
for (int m = 0; m < NUM_MASTERS; m++) begin
    s_axis_tready[m] = 1'b0;
    for (int s = 0; s < NUM_SLAVES; s++)
        if (grant_matrix[s][m])
            s_axis_tready[m] = m_axis_tready[s];
end
```

This preserves AXI-Stream ready/valid handshake semantics across the switch.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

17 Boundary Module Instantiation

Delta does not instantiate external boundary modules for the flat crossbar. The `DeltaFlatCrossbar` class directly emits the full crossbar logic using the `Module` framework. However, the generator does produce small protocol-agnostic primitives when tree generation is requested.

17.1 Flat crossbar

The entire crossbar is self-contained in `delta_axis_flat_<M>x<N>.sv`:

- No external arbiter module is instantiated; arbitration is emitted inline.
- No external FIFOs are used; the design is purely combinational between registered arbitration state.

17.2 Tree primitives

When `--nodes` is passed, `delta_split_1to2.sv` is emitted by the same generator. The `complete_tree_generator.py` script emits additional primitives:

Primitive	Purpose
`delta_split_1to2`	Routes one AXI-Stream input to two outputs based on a `TDEST` bit.
`delta_merge_2to1`	Arbiters two inputs to one output with packet atomicity.
`delta_fanout_1to<N>`	Schematic fan-out tree using splitters (partial implementation).
`delta_fanin_<N>to1`	Schematic fan-in tree using mergers (partial implementation).

The tree primitives are not yet production-complete; they exist as building blocks and documentation of the intended topology.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

18 Directory Layout

```
delta/
├── bin/
│   ├── delta_generator.py            # framework-based flat crossbar
generator
│   ├── complete_tree_generator.py    # tree primitive generator
│   ├── delta_generator_v1_backup.py  # legacy generator (reference
only)
│   └── delta_performance_model.py    # performance estimation tool
├── rtl/
│   └── delta_axis_flat_<M>x<N>.sv    # generated flat crossbar
├── dv/
│   └── tests/                        # cocotb test suite
├── PRD.md                            # product requirements
└── CLAUDE.md                         # AI development guide
```

The generator writes directly into the directory specified by `--output-dir` (default `../rtl`). Unlike the APB crossbar, Delta does not currently maintain a batch script with a fixed variant table; callers generate the sizes they need.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

19 Module Header and Parameters

The `Module` framework emits the module header automatically from the parameter and port strings added by `DeltaFlatCrossbar.__init__`.

19.1 Header comment

The generated file begins with a block comment documenting the configuration:

```
//
================================================================================
========
// Module: delta_axis_flat_2x4
// Project: Delta - AXI-Stream Crossbar Generator
//================================================================================
==========
// Description: AXI-Stream 2×4 Flat Crossbar Switch
//
// Configuration:
//   - Topology: flat
//   - Masters: 2
//   - Slaves: 4
//   - Data Width: 64 bits
//   - TDEST Width: 2 bits
//   - TID Width: 1 bits
//   - Arbiter: round_robin
//
// Features:
//   - Per-slave round-robin arbitration
//   - Packet atomicity (locked grant until TLAST)
//   - Registered outputs for timing closure
//   - Full AXI-Stream compliance
//================================================================================
==========
```

19.2 Port naming convention

Direction	Naming
Master-side inputs	`s_axis_<signal>[NUM_MASTERS]`
Master-side outputs	`s_axis_tready[NUM_MASTERS]`
Slave-side outputs	`m_axis_<signal>[NUM_SLAVES]`
Slave-side inputs	`m_axis_tready[NUM_SLAVES]`

Optional signals (`tstrb`, `twakeup`, the `t*chk` check signals and the `s_/m_parity_error` flags) are declared only when enabled. With `--amba5` the header also gains derived check-width parameters (`TDATACHK_W = DATA_WIDTH / 8` and so on), and the module instantiates `rtl/common/dataint_parity`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

20 Flat Crossbar

The flat crossbar is the production artifact. It is a single module that connects any master to any slave based on TDEST.

20.1 Internal structures

Signal / Variable	Purpose
`request_matrix[NUM_SLAVES]`	One-hot request vector per slave.
`grant_matrix[NUM_SLAVES]`	One-hot grant vector per slave.
`last_grant[NUM_SLAVES]`	Round-robin pointer per slave.
`packet_active[NUM_SLAVES]`	Lock grant until `TLAST`.

20.2 Optional features

20.2.1 Performance counters

When `add_performance_counters=True`, the generator emits 32-bit packet counters:

```
logic [31:0] pkt_count_master [NUM_MASTERS];
logic [31:0] pkt_count_slave [NUM_SLAVES];
```

Counters increment on `valid && ready && tlast` at the master or slave interface.

20.2.2 Assertions

Under `` `ifdef FORMAL ``:

- One-hot grant check per slave.
- TDEST range check per master.

These assertions are intended for formal verification flows and are stripped during normal simulation if `FORMAL` is not defined.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

21 Tree Primitives

Delta includes scaffolding for tree topologies in addition to the flat crossbar. These primitives are generated separately and are intended for future hierarchical switches.

21.1 delta_split_1to2

Generated by `delta_generator.py --nodes`. It routes one AXI-Stream input to two outputs based on a configurable TDEST bit:

```
assign route_to_1 = s_axis_tdest[SPLIT_BIT];
assign m0_axis_tvalid = s_axis_tvalid && !route_to_1;
assign m1_axis_tvalid = s_axis_tvalid && route_to_1;
assign s_axis_tready  = route_to_1 ? m1_axis_tready : m0_axis_tready;
```

21.2 delta_merge_2to1

Generated by `complete_tree_generator.py --type merger`. It arbitrates two inputs to one output using round-robin with packet atomicity.

21.3 Fan-out / fan-in trees

`complete_tree_generator.py --type fanout --size N` and `--type fanin --size N` emit structural scaffolds for cascaded splitters and mergers. The implementations are currently partial; they illustrate the intended topology and are not yet part of the regression baseline.

21.4 Filelist convention

A complete Delta subsystem typically needs only:

`rtl/delta_axis_flat_<M>x<N>.sv`

Tree-based systems add the appropriate splitter/merger files. The component-level `Makefile` or area flow maintains the canonical filelist.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

22 Full Regeneration Rule

Any change to the files listed below must be followed by regeneration of the affected fixtures and a full regression run.

Source	Why it forces regeneration
`bin/delta_generator.py`	All flat crossbar output comes from this file.
`bin/complete_tree_generator.py`	All tree primitive output comes from this file.
`bin/rtl_generators/verilog/module.py`	Framework changes affect header/footer formatting.

22.1 Regeneration command

```
cd projects/components/fabric-gen-ip/delta/bin
python delta_generator.py --masters 2 --slaves 4 --output-dir ../rtl
python complete_tree_generator.py --type merger --output ../rtl
```

After regeneration, the only expected `git diff` should be in `rtl/`.

22.2 Do not hand-edit generated files

The `.sv` files in `rtl/` are generator output. Hand-edits will be overwritten on the next regeneration. Fix the generator instead.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

23 Pre-Generated Fixtures

Delta does not currently ship a fixed family of pre-generated sizes. The recommended baseline fixtures are generated on demand with:

```
python delta_generator.py --masters 2 --slaves 4 --data-width 64 --
output-dir ../rtl
python delta_generator.py --masters 4 --slaves 4 --data-width 64 --
output-dir ../rtl
python delta_generator.py --masters 2 --slaves 4 --data-width 64 --
amba5 --output-dir ../rtl
```

If the project later adopts a canonical fixture set, add a `generate_all.py` batch script and document the fixed sizes here.

23.1 Custom variants

Custom sizes are fully supported. When generating a custom variant, record the exact command in the subsystem integration notes so the fixture can be reproduced.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

24 DV Suite

The component-level DV suite lives in `dv/tests/` and uses cocotb.

24.1 Test focus areas

A complete Delta DV suite should cover:

Scenario	What to verify
Single-packet routing	A master sends a packet to each slave; `TDEST` selects correctly.
Multi-master contention	Two masters target the same slave; arbitration is fair and packet-atomic.
Backpressure	Assert `m_axis_tready` low; `s_axis_tready` follows correctly.
TLAST handling	Grants release only after `TLAST`.
AMBA 5 sidebands	`TWAKEUP` steers by `TDEST`. Payload `*CHK` is carried with `TDATA`. `TVALIDCHK`/`TWAKEUPCHK`/`TREADYCHK` are the inversions of their signals. A bad check sets the right sticky flag.
Out-of-range TDEST	Formal assertion fires; simulation drops the request.
Performance counters	Counters increment on `valid && ready && tlast`.

24.2 Running the suite

```
cd projects/components/fabric-gen-ip/delta/dv/tests
make clean-all && make run-all-func-parallel
```

Today the suite is one file, `test_delta_axis_flat_2x4_strb_amba5.py`, covering the AMBA 5 row above against `rtl_test/delta_axis_flat_2x4_strb_amba5.sv`. It has three tests at gate, func and full. The other rows are still open for the base flavor.

After a generator change, this must pass with zero failures before the regenerated RTL is committed.

24.3 Adding a new test

1. Add a new cocotb test file in `dv/tests/`.
2. Reference the generated module under test.
3. Add the test name to the area regression list.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

25 Lint Checks

The generated RTL must pass the project’s lint flow.

25.1 Common concerns

Concern	Mitigation in generator
Unpacked array lint	Arrays are declared as `logic [W-1:0] name [N]`; ensure the lint tool supports unpacked array ports.
Width mismatches	`DEST_WIDTH` and `ID_WIDTH` are computed from slave/master counts.
Latch inference	All `always_comb` blocks assign all outputs; no incomplete cases.
Uninitialized logic	Default assignments precede conditional overrides in every mux.

25.2 Synthesis smoke test

Run a synthesis tool on the 2×4 variant after any generator change. The expected result is clean elaboration with no combinational loops and no latch inference.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

26 Adding a New Topology

To add a new topology beyond `flat`:

1. Add a new class in `delta_generator.py` (e.g., `DeltaTreeCrossbar`).
2. Derive it from `Module` or reuse `DeltaFlatCrossbar` as a building block.
3. Add the topology name to the `--topology` choices.
4. In `main()`, instantiate the new class when the selected topology matches.
5. Add a DV test that exercises the new topology.
6. Regenerate fixtures and run the full DV suite.

26.1 Tree topology notes

The existing `DeltaSplitter1to2` and `complete_tree_generator.py` provide the building blocks. A full tree crossbar would recursively instantiate splitters and mergers to build a multi-stage network. Because tree routing uses bit slices of `TDEST`, ensure `DEST_WIDTH` is at least as large as the tree depth.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

27 Adding a New Generator Option

To add a scalar option:

1. Add the field to `DeltaConfig`.
2. Add the CLI argument to `main()` in `delta_generator.py`.
3. Pass the argument into `DeltaConfig(...)`.
4. Use the config field inside the appropriate `generate_*` method.
5. Document the option in this GAS, the HAS, and the PRD if it changes hardware behavior.
6. Add validation in `main()` or inside the generator method.
7. Add a DV test that exercises the new option.

27.1 Backward compatibility

Default values must preserve existing generated output. If the new option changes the default port list, introduce it as an opt-in flag first.

27.2 Example: adding a QoS sideband

```
@dataclass
class DeltaConfig:
    ...
    qos_width: int = 0
```

Then emit `s_axis_tqos` / `m_axis_tqos` arrays and mux them in `generate_crossbar_mux()`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

28 Adding AMBA 5 Sidebands

The current AMBA 5 support covers TWAKEUP and the full IHI 0051B chapter 5 check-signal set (TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK). To add more AMBA 5 sidebands:

1. Add the new signal to `DeltaConfig` (e.g., a width field).
2. In `DeltaFlatCrossbar.__init__`, add the new port declarations when `amba5` is enabled.
3. Add routing logic in a new `generate_*_logic()` method.
4. Call the new method from `verilog()`.
5. If the new signal is covered by a check signal (a new payload field gets a `*CHK` per Table 5-1), add it to `_payload_chk_fields()`. That one list drives the ports, the carry mux, the ingress checkers and the error OR.
6. Update the DV suite (`dv/tests/test_delta_axis_flat_2x4_strb_amba5.py` and its harness) to exercise the new sideband.

28.1 TWAKEUP design note

`TWAKEUP` is a hint, not a data beat. It is routed by `TDEST` without arbitration because it may precede `TVALID`. If a future sideband requires grant-based routing, follow the payload check-bit pattern in `generate_crossbar_mux()`.

28.2 Check-signal design note

The payload check bits ride the same grant as TDATA and are carried, not regenerated, so a beat corrupted upstream still fails downstream. The crossbar also checks them on ingress (enabled by TVALID). It generates the single-bit checks for the signals it drives (TVALIDCHK and TWAKEUPCHK on slave ports, TREADYCHK on master ports) and checks the ones it receives. Every bit is one `rtl/common/dataint_parity` instance, odd parity, CHUNKS=1. A new check must follow the same rule: odd parity, one bit per 8 bits, top bit covers the remainder (IHI 0051B section 5.3).

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

29 Testing an Extension

Every generator change must be exercised by at least one generated fixture and one DV test.

29.1 Minimal test plan

1. **Generator unit test** — call `DeltaFlatCrossbar(cfg).verilog()` with the new option and verify the emitted string contains the expected constructs.
2. **Fixture regeneration** — generate the affected fixtures and inspect the diff.
3. **Simulation test** — add or extend a cocotb test that exercises the new behavior.
4. **Lint/synthesis smoke** — run the lint flow on the affected variant.

29.2 Example: adding a custom variant test

```
# dv/tests/test_delta_flat_4x4.py
import cocotb

class TestDeltaFlat4x4:
    @cocotb.test()
    async def test_basic_routing(self, dut):
        # exercise all 4 masters against all 4 slaves
```

The test must be added to the `run-all-gate` target or an equivalent regression list.

29.3 Commit discipline

Commit generator source changes, regenerated RTL, and DV tests in a single logical change. Do not commit hand-edited generated files.